哈爾濱工程大學

第 14 讲 / 全 17 讲

C 语言字符串处理详解

从手写遍历到 string.h 工具箱

strlen
数一数有几个有效字符

strcpy
把一串内容抄到另一串

strcmp
比大小、判相等

逐字符处理
一个一个字符地看、改、数

主讲教师：郎大鹏

本节目标

学完你能做 5 道综合题

哈尔滨工程大学

1. 说清字符串以 '\0' 结尾的含义，分清 strlen 与 sizeof
2. 会用 strlen / strcpy / strcat 三个库函数，记得写头文件
3. 讲明白为什么不能给字符串用 = 赋值，也不能用 == 比较
4. 会用 strcmp 比大小，说得清返回 0、正数、负数各是什么意思
5. 能手写循环一个字符一个字符地处理：转大小写、分类统计
6. 独立完成逆序、回文、单词计数、查子串、密码强度五道题

01

字符串库函数

以前全靠手写遍历，现在有现成工具： strlen / strcpy / strcat / strcmp

先会用，再弄懂它们替我们写了哪段循环

哈爾濱工程大学

字符串处理的现状

回忆数组课

哈尔滨工程大学

```
char s[20] = "hello";
int i = 0;
//  想知道长度？自己数
while (s[i] != 0) {
    i++;
}
printf("%d", i);    // 5
//  想复制？再写一段循环
//  想比较？再写一段循环
//  想拼接？还得再写一段
//  四个动作  =  四段一样的循环
//  数组课咱们就是这么干的
```

手工时代

求长度、复制、比较、拼接，四件事要写四段几乎一模一样的循环，改一个字符就出错。

现在有工具

C 语言把这些常用循环写好了，放在 string.h 里，我们直接喊名字用就行。

但原理没变

库函数内部也是一个字符一个字符地走到结尾，只是这次不用你亲手写。

strlen ：数有几个字符

string.h 第 1 件

哈爾濱工程大學

```c
#include <string.h>  //  先打招呼
char s[20] = "hello";
int n = strlen(s);
printf("%d", n);       // 5
printf("%d", strlen("hi"));  // 2
char t[20] = "a b";
printf("%d", strlen(t));  // 3
char e[20] = "";
printf("%d", strlen(e));  // 0
//  一路数到 '\0' 就停下
//  结尾那个 '\0' 自己不算数
//  存 hello 要 6 个字符的房间
```

strlen 干什么

从第一个字符开始数，遇到结尾标记就停，返回数过的字符个数。

不含结尾标记

"hello" 是 5 个字符，strlen 就返回 5 ；空格、标点都算一个字符。

用法记三句

strlen(字符串) 得到整数；空串得到 0 ；结果可以直接放进 printf 和 if 里用。

手写求长度 vs strlen

回收数组课的伏笔

哈尔滨工程大学

数组课的写法：自己写循环

```
char s[20] = "hello";
int len = 0;
int i;
for (i = 0; s[i] != 0; i++) {
    len++;
}
printf("%d", len);  // 5
// 6 行代码只干一件事
// 换个数组还得再写一遍
```

现在：一句话调用 strlen

```
#include <string.h>
char s[20] = "hello";
int len;
len = strlen(s);
printf("%d", len);  // 5
if (strlen(s) > 3) {
    printf("long");
}
// 一句话搞定，换串照样用
```

同样的循环，不同的写法 strlen 干的事和我们手写的一模一样，只是循环搬到了库里

strcpy ：复制字符串

string.h 第 2 件

哈爾濱工程大學

```c
#include <string.h>
char s1[20] = "hello";
char s2[20];
strcpy(s2, s1);   //  抄过去
printf("%s", s2); // hello
//  抄的是内容，不是地址
//  改  s2  不会动  s1
s2[0] = 'H';
printf("%s", s1); // hello
printf("%s", s2); // Hello
//  错误写法：  s2 = s1;
//  数组名是常量，不能赋值
```

strcpy 干什么

把第二个字符串的内容，一个字符一个字符抄进第一个数组，结尾标记一起抄过去。

为什么不能 s2 = s1

数组名是房间号常量，赋值号左边必须是能变的房间，常量站左边编译就报错。

目标要装得下

s2 的格子数必须不少于 s1 长度加一，抄不下就会写出界，程序当场崩。

strcat ：拼接字符串

string.h 第 3 件

哈爾濱工程大學

```c
#include <string.h>
char a[40] = "hello";
char b[20] = "world";
strcat(a, b);
printf("%s", a);   // helloworld
// 从 a 的结尾处接着写
char c[40] = "hi";
strcat(c, " there");
printf("%s", c);   // hi there
printf("%d", strlen(c));  // 8
// a 留了 40 格，装得下
// 若 a 只有 6 格，就写出界
```

strcat 干什么

把第二个字符串接到第一个字符串的尾巴上，拼好的结果仍然放在第一个数组里。

从哪里开始接

先找到第一个串的结尾标记，从那一格开始往后写，写完再补一个新的结尾标记。

第一个串要够大

格子数要不少于 原来长度 + 新串长度 + 1 ，所以上面直接开成 40 格。

用之前先打招呼

速查表

哈爾濱工程大學

```
#include <stdio.h>
#include <string.h>   //  字符串工具箱在这
```

strlen(s)

数有效字符，不含结尾标记

```
int n = strlen(s);  // 得 5
```

返回 int ，空串得 0

strcpy(a, b)

把 b 的内容抄进 a

```
strcpy(a, b);
```

a 要装得下 b 才行

strcat(a, b)

把 b 接到 a 的尾巴上

```
strcat(a, "ok");
```

结果留在 a 里， a 要够大

strcmp(a, b)

比大小，相等时返回 0

```
if (strcmp(a, b) == 0)
```

下一部分细讲

02

比较与逐字符处理

strcmp 到底怎么比 · 为什么 == 不能用 · 转大小写 · 逐字符统计

把字符串拆成一个一个字符，想怎么改就怎么改

哈爾濱工程大学

strcmp ：两个串怎么比

string.h 第 4 件

哈尔滨工程大学

```
char a[20] = "hello";
char b[20] = "hero";
int r = strcmp(a, b);
printf("%d", r);   //  负数
//  第 1 个: h 对 h，一样
//  第 2 个: e 对 e，一样
//  第 3 个: l 对 r，l 更小
//  所以 hello 小于 hero
printf("%d", strcmp("ab", "ab"));  // 0
printf("%d", strcmp("ab", "ac"));  //  负
printf("%d", strcmp("ac", "ab"));  //  正
```

比的是编号

两个串从头开始，一个对一个地比字符的 ASCII 编号，不是比长短。

什么时候停

第一次分出大小就停下：前者大返回正数，前者小返回负数；比完都一样才是 0 。

hero 与 hello

前两个字母都一样，第三个 l 排在 r 前面，所以 hello 比 hero 小。

为什么不能用 == 比

回忆指针课

哈尔滨工程大学

```
char a[20] = "hero";
char b[20] = "hero";
if (a == b) {       //  错!
    printf("same");
} else {
    printf("no");  //  走这里
}
// a  和  b  是两间不同的房
// ==  比房间号，必然不等
if (strcmp(a, b) == 0) {
    printf("same");  //  走这里
}
```

== 比的是地址

数组名就是这串字符的首地址， == 左右两边比的是两个地址，跟内容一点关系没有。

所以永远不等

两个数组哪怕内容一模一样，也是两间不同的房，房间号不可能相同。

正确写法

想知道内容是否相同，就写 strcmp(a, b) == 0 ；想知道谁大谁小，就看它是正还是负。

大小写转换


```
char s[20] = "Hello";
int i;
for (i = 0; s[i] != 0; i++) {
    if (s[i] >= 'a' && s[i] <= 'z') {
        s[i] = s[i] - 32;
    }
}
printf("%s", s);   // HELLO
// ctype.h 里有现成的
s[0] = toupper(s[0]);
s[1] = tolower(s[1]);
printf("%s", s);   // hELLO
```

大小写差 32

同一个字母的大小写编号相差 32：小写减 32 变大写，大写加 32 变小写。

先判断再改

只改落在 a 到 z 之间的字符，别把数字和空格也减掉 32。

库里也有

toupper / tolower 一次转一个字符，用之前记得 #include <ctype.h>。

逐字符分类统计


```
char s[40] = "ab12 cd";
int dig = 0, let = 0, oth = 0;
int i;
for (i = 0; s[i] != 0; i++) {
    if (s[i] >= '0' && s[i] <= '9') {
        dig++;
    } else if (s[i] >= 'a' && s[i] <= 'z') {
        let++;
    } else {
        oth++;
    }
}
```

三个计数器

准备三个变量，从头到尾走一遍，遇到什么就给对应的那个加一。

按区间判断

数字落在 0 到 9 之间，小写字母落在 a 到 z 之间，剩下的统统算其他。

结果：2 / 4 / 1

数字 2 个、字母 4 个、其他 1 个（那个空格）。ctype.h 的 isdigit 也能判。

字符串作函数参数

呼应指针课

哈尔滨工程大学

```c
void upper(char *s) {
    int i;
    for (i = 0; s[i] != 0; i++) {
        s[i] = toupper(s[i]);
    }
}
// 写成 char s[] 效果一样
int main(void) {
    char s[20] = "hello";
    upper(s);       // 只写名字
    printf("%s", s);   // HELLO
}
```

形参两种写法

char s[ ] 和 char *s 是同一件事：收到的都是这串字符的首地址。

调用只写名字

实参写 s 就够了，既不写方括号，也不用加取地址符，数组名本身就是地址。

改了外面跟着变

传进去的是地址，函数里改哪一格，main 里那一格就真的变了——和指针课一个道理。

strlen 与 sizeof 的坑

结尾标记占一格

```
char s[20] = "hello";
printf("%d", strlen(s));  // 5
printf("%d", sizeof(s));  // 20
// strlen 数有效字符
// sizeof 数一共开了多少格
char t[] = "hi";
printf("%d", strlen(t));  // 2
printf("%d", sizeof(t));  // 3
// 3 = h i 再加一个结尾标记
char u[5] = "hello";  // 装不下
scanf("%s", s);  // 遇空格就停
```

strlen 数内容

从第一格走到结尾标记，数的是真正写进去的字符，换个内容结果就变。

sizeof 数容量

算的是这个数组一共占多少字节，跟你放了多少字符没关系，永远是 20 。

两个都要留神

开数组要给结尾标记多留一格； scanf 配 %s 只读一个单词，空格后面读不到。

03

字符串应用

逆序 · 回文 · 单词计数 · 查子串 · 字符串数组 · 密码强度

把前面学的循环、下标、库函数一起用起来

哈爾濱工程大学

字符串逆序

呼应排序课的交换

哈尔滨工程大学

```
char s[20] = "hello";
int i = 0, j, t;
j = strlen(s) - 1;  //  最后一个
while (i < j) {
    t = s[i];
    s[i] = s[j];
    s[j] = t;
    i++;
    j--;
}
printf("%s", s);    // olleh
```

两个下标夹逼

i 从 0 往后走，j 从最后一个字符往前走，两人碰头就结束。

三步交换

借一个临时变量 t：先存、再盖、最后填回去，和排序课换两个数一模一样。

原地完成

不用另开一个数组，就在原来的数组里换，结尾标记始终待在最后不动。

回文判断

正读反读一样

```
char s[20] = "level";
int i = 0, j = strlen(s) - 1;
int ok = 1;    //  先假设是
while (i < j) {
    if (s[i] != s[j]) {
        ok = 0;    //  有一对不同
        break;     //  提前收工
    }
    i++;   j--;
}
printf("%d", ok);    // 1  是回文
```

头尾对着看

第 1 个对最后 1 个，第 2 个对倒数第 2 个，成对地比，比到中间为止。

不必真逆序

只要有一对不一样就能下结论，用 break 提前跳出，省掉后面的比较。

flag 变量套路

先假设成立，找到反例就改成 0——这个 " 先假设再打脸 " 的写法以后到处都用。

单词计数

连续空格是个坑

```
char s[40] = "hi  tom";
int i, cnt = 0, sp = 1;
// sp=1：前面是空格或开头
for (i = 0; s[i] != 0; i++) {
    if (s[i] != ' ') {
        cnt = cnt + sp;
        sp = 0;
    } else {
        sp = 1;
    }
}
printf("%d", cnt);    // 2
```

朴素想法

先数空格，再加 1 ：看起来对，其实只在单词之间恰好一个空格时才成立。

连续空格的坑

"hi tom" 中间两个空格，数空格加 1 会得到 3 ，实际只有 2 个单词。

记住上一位

用 sp 记着 " 上一位是不是空格 " ，只有从空格跨到非空格的那一刻才算一个新词。

查找子串

手写 strstr 的思想

哈爾濱工程大學

```
char s[40] = "hello world";
char t[10] = "wor";
int i, j, pos = -1;
for (i = 0; s[i] != 0; i++) {
    j = 0;
    while (t[j] != 0 && s[i+j] == t[j]) {
        j++;
    }
    if (t[j] == 0 && pos < 0) {
        pos = i;    //  起点是  6
    }
}
```

枚举起点

外层的 i 从第 0 格一直试到结尾：把每个位置都当成 " 可能的开头 " 试一遍。

逐个比对

里层拿 s[i+j] 和 t[j] 对着比，只要短串被比完，说明从这个起点整段对上了。

库里有现成的

string.h 里的 strstr 就是这个思路，找不到时返回空地址； pos 保持 -1 表示没找到。

字符串数组

多个串一起管


```c
char name[3][10] = {"Tom", "Bob", "Li"};
int i;
for (i = 0; i < 3; i++) {
    printf("%s", name[i]);
}
// name[0] 就是第 1 个串
printf("%c", name[1][0]);   // B
// 第 2 行的第 1 个字符
if (strcmp(name[0], name[1]) > 0) {
    printf("Tom 排后面");
}
// 每行 10 格，库函数照样用
```

一行放一个串

二维字符数组：第一个下标选第几个串，第二个下标选这个串里的第几个字符。

每行长度固定

每行都是 10 格，多出来的空着不用。开的时候按最长的那个串留够格子。

库函数照样能用

name[i] 就是一个串，strlen、strcpy、strcmp 都可以直接拿它当参数。

密码强度检查

综合小项目

哈尔滨工程大学

```
char p[30] = "ab12xy";
int i, dig = 0, let = 0;
int n = strlen(p);    //  先看长度
for (i = 0; i < n; i++) {
    if (p[i] >= '0' && p[i] <= '9') {
        dig++;    //  数数字
    }
    if (p[i] >= 'a' && p[i] <= 'z') {
        let++;    //  数字母
    }
}
printf("%d", n >= 6 && dig && let);
```

拆成三个条件

长度够不够 6 位、有没有数字、有没有字母，一条一条查，最后一起判断。

边走边数

整串只走一遍：每看到一个字符就问它是数字还是字母，给对应的计数器加一。

同时成立才算过

三个条件用 && 串起来，全为真才打印 1；想加 " 必须有大写 " ，再添一个计数器。

例 1：字符串逆序

例题精讲

哈尔滨工程大学

题目 把字符串 "hello" 原地倒过来，变成 "olleh"，不许另开数组

```
char s[20] = "hello";
int i = 0, j, t;
j = strlen(s) - 1;    //  指向最后
while (i < j) {
    t = s[i]; s[i] = s[j]; s[j] = t;
    i++;  j--;
}
printf("%s", s);    // olleh
// 5 个字符只需换 2 次
//  中间那个字符原地不动
```

运行结果

```
olleh
```

· i 从头往后，j 从尾往前

· 每轮换一对，然后两头各走一步

· i 与 j 相遇（i < j 不成立）就收工

· 结尾标记一直留在最后，不用管它

· 偶数长度换 n/2 次，奇数长度换 (n-1)/2 次

例 2：回文判断

例题精讲

哈尔滨工程大学

题目 判断 "level" 是不是回文：正着读和反着读完全一样

```
char s[20] = "level";
int i = 0, j, ok = 1;
j = strlen(s) - 1;
while (i < j) {
    if (s[i] != s[j]) {
        ok = 0;  break;
    }
    i++;  j--;
}
printf("%d", ok);    // 1  是回文
```

运行结果

1　　//　是回文

· l 对 l、e 对 e，都比完了 ok 还是 1

· 换成 "hello"：h 对 o 就不同，ok 变 0

· 奇数长度时，中间那个字符不用比

· 想忽略大小写？比之前先统一转成小写

· 判断串相等用 strcmp，判断字符相等用 ==

例 3：统计单词个数

例题精讲

哈尔滨工程大学

题目 统计 "hi tom and jerry" 里有几个单词，注意中间连着两个空格

```
char s[40] = "hi  tom and jerry";
int i, cnt = 0, sp = 1;
// sp = 1：前面是空格或开头
for (i = 0; s[i] != 0; i++) {
    if (s[i] != ' ') {
        cnt = cnt + sp;  sp = 0;
    } else {
        sp = 1;
    }
}
printf("%d", cnt);    // 4
```

运行结果

```
4     // hi tom and jerry
```

- sp 一开始给 1，保证第一个词算进去
- 连着两个空格：第二个空格不再加一
- 末尾若还有空格，也不会多算一个
- 数空格再加 1 的写法在这里会得到 5
- 想读一整行带空格的句子，%s 做不到

例 4：查找子串位置

例题精讲

哈尔滨工程大学

题目 在 "hello world" 里找 "wor" 第一次出现的位置，找不到就报 -1

```
char s[40] = "hello world";
char t[10] = "wor";
int i, j, pos = -1;
for (i = 0; s[i] != 0; i++) {
    for (j = 0; t[j] != 0; j++) {
        if (s[i+j] != t[j]) {  break;  }
    }
    if (t[j] == 0 && pos < 0) {  pos = i;  }
}
printf("%d", pos);    // 6
```

运行结果

```
6     // 从 0 开始数
```

- 外层 i 试遍每个可能的开头
- 内层 j 从头比到尾，中间断了就 break
- j 走到 t 的结尾，说明这一段整段对上
- pos 保持 -1 就表示压根没找到
- 库函数 strstr 干的也是这件事

例 5 ：密码强度检查

例题精讲

哈爾濱工程大學

题目 密码不少于 6 位、至少含 1 个数字和 1 个字母，才算合格

```
char p[30] = "ab12xy";
int i, dig = 0, let = 0;
int n = strlen(p);    // 先看长度
for (i = 0; i < n; i++) {
    if (p[i] >= '0' && p[i] <= '9') { dig++; }
    if (p[i] >= 'a' && p[i] <= 'z') { let++; }
}
// 够长 + 有数字 + 有字母
printf("%d", n >= 6 && dig && let);
// 换成 "abc" 只有字母，得 0
```

运行结果

```
1      // ab12xy 合格
```

- dig 数出 2 个数字，let 数出 4 个字母
- 三个条件用 && 串起来，缺一个都不行
- "abc" 长度不够且没数字，结果是 0
- "123456" 只有数字，结果也是 0
- 想加 " 必须含大写 " ？再添一个计数器

常见错误盘点

这六条最常踩

哈尔滨工程大学

- ✗ 用 == 比较两个字符串：比的是两个地址，不是内容，要用 strcmp
- ✗ strcpy 的目标数组开得太小：抄不下就写出界，程序当场崩
- ✗ strcat 前没给目标留够地方：要 原串长度 + 新串长度 + 1 才够
- ✗ strlen 与 sizeof 混着用：一个数的是内容，一个数的是容量
- ✗ 开数组忘了给结尾标记留一格： n 个字符得开 n + 1 格
- ✗ strcmp 的返回值看反：相等时返回 0 ，不相等才返回非零

小结

- string.h 四件套：strlen 求长、strcpy 复制、strcat 拼接、strcmp 比较
- 字符串不能用 = 赋值，也不能用 == 比较，那是地址不是内容
- 逐字符处理就是循环加下标：转大小写、分类统计、逆序、回文全靠它
- 结尾标记占一格，开数组时永远多留一个位置

下一讲预告

结构体——把不同类型的数据捆在一起

哈爾濱工程大学